

CODE:

INDEX / TABLE OF CONTENTS

- 1. Cloud Spanner Configuration**
- 2. Publisher.py**
- 3. clean_ecom2.py — PySpark Transformation**
- 4. Airflow DAG Definition — ecom_pipeline_dag.py**
 - 4.1 Dataproc Serverless Configuration**
 - 4.2 BigQuery Federated Aggregation**
- 5. BigQuery Verification SQL**
- 6. IAM Roles & Permissions Summary**

1.Spanner:

Database name: ecom-db

Database dialect: Google Standard SQL

```
CREATE TABLE EcomOrders (  
    order_id STRING(64) NOT NULL,  
    product_id STRING(64) NOT NULL,  
    user_id STRING(64),  
    quantity INT64,  
    order_date TIMESTAMP,  
) PRIMARY KEY (order_id, product_id);
```

2.Publisher.py:

Open the cloud shell and create the publisher.py file and save it then run it `python3 publisher.py`

```
import json  
import time  
import random  
import requests  
from google.cloud import pubsub_v1  
  
# PROJECT & TOPIC CONFIGURATION  
PROJECT_ID = "snappy-mapper-498509-e0"  
TOPIC_ID = "ecom-orders-topic"  
  
publisher = pubsub_v1.PublisherClient()  
topic_path = publisher.topic_path(PROJECT_ID, TOPIC_ID)  
  
# Public E-Commerce API (Random Carts/Orders)
```

```
API_BASE_URL = "https://fakestoreapi.com/carts"

def fetch_and_publish():
    try:
        # Fetch a random cart order (ID 1 through 7)
        cart_id = random.randint(1, 7)

        response =
requests.get(f"{API_BASE_URL}/{cart_id}").json()

        # Add an ingestion timestamp
        response["ingested_at"] = time.strftime("%Y-%m-
%dT%H:%M:%SZ", time.gmtime())

        payload = json.dumps(response).encode("utf-8")
        future = publisher.publish(topic_path, payload)
        print(f"Published Order Cart ID #{cart_id} to Pub/Sub.
Message ID: {future.result()}")
    except Exception as e:
        print(f"Error publishing order: {e}")

if __name__ == "__main__":
    print("Starting E-Commerce Order Streamer to Pub/Sub...
Press Ctrl+C to stop.")
    while True:
        fetch_and_publish()
        time.sleep(5) # Fetch new order every 5 seconds
```

3.clean_ecom.py: (Location- gs://pub-sub-cleandata-dag/scripts/clean_ecom2.py)

This file upload in the gcs bucket which we create `pub-sub-cleandata` in the create a folder `scripts/` in that upload this file.

```
import sys

from pyspark.sql import SparkSession

from pyspark.sql.functions import col, to_timestamp

# Initialize Spark Session
spark = SparkSession.builder \
    .appName("CleanEcomData2") \
    .getOrCreate()

try:

    gcs_input_path = "gs://pub-sub-dag-rawdata/*.json"
    print(f"Reading raw JSON data from: {gcs_input_path}")

    # 1. Read Raw JSON from GCS Bronze Bucket
    raw_df = spark.read.json(gcs_input_path)

    raw_count = raw_df.count()
    print(f"Total raw records read from GCS: {raw_count}")

    if raw_count == 0:

        raise ValueError("CRITICAL ERROR: No records found in
GCS input path or JSON files are empty!")

    # Print raw schema to inspect JSON structure in logs
    raw_df.printSchema()
```

```

# 2. Clean, Cast Types, and Deduplicate
cleaned_df = raw_df.select(
    col("order_id").cast("string"),
    col("product_id").cast("string"),
    col("user_id").cast("string"),
    col("quantity").cast("long"),
    to_timestamp(col("order_date")).alias("order_date")
).dropDuplicates(["order_id"])

cleaned_count = cleaned_df.count()

print(f"Total cleaned & deduplicated records to write:
{cleaned_count}")

if cleaned_count == 0:
    raise ValueError("CRITICAL ERROR: All records were
dropped during cleaning/deduplication!")

# 3. Save Parquet to Silver GCS Bucket
parquet_path = "gs://pub-sub-cleandata-dag/parquet/"
print(f"Writing Parquet data to: {parquet_path}")
cleaned_df.write.mode("overwrite").parquet(parquet_path)

# 4. Write directly to Cloud Spanner
print("Writing data to Cloud Spanner table
'EcomOrders'...")

cleaned_df.write \
    .format("cloud-spanner") \
    .option("projectId", "snappy-mapper-498509-e0") \
    .option("instanceId", "ecom-spanner-instance") \

```

```

        .option("databaseId", "ecom-db") \
        .option("table", "EcomOrders") \
        .mode("append") \
        .save()

    print("Successfully processed and loaded data to Cloud
    Spanner and GCS.")

except Exception as e:

    print(f"JOB FAILED WITH ERROR: {str(e)}", file=sys.stderr)

    raise e # Fail the Dataproc batch so Airflow knows the
    task failed!

finally:

    spark.stop()

```

4. Airflow DAG Definition (ecom_pipeline_dag.py)

Location: Cloud Composer /dags folder

```

from datetime import datetime, timedelta

from airflow import DAG

from airflow.providers.google.cloud.operators.dataproc import
DataprocCreateBatchOperator

from airflow.providers.google.cloud.operators.bigquery import
BigQueryInsertJobOperator

# Default arguments for DAG execution
default_args = {

    'owner': 'data-engineering',

    'depends_on_past': False,

    'start_date': datetime(2026, 1, 1),

    'retries': 1,

    'retry_delay': timedelta(minutes=5),

```

```

}

# Define DAG pipeline
with DAG(
    'ecom_data_pipeline',
    default_args=default_args,
    description='End-to-End Pipeline: Dataproc Serverless
cleans GCS raw JSONs to Spanner & BigQuery aggregates via
External Connection',
    schedule_interval='0 0 * * *',
    catchup=False,
) as dag:
    # Task 1: Dataproc Serverless PySpark Batch Job
    run_dataproc_batch = DataprocCreateBatchOperator(
        task_id='run_dataproc_pyspark_batch',
        project_id='snappy-mapper-498509-e0',
        region='asia-south1',
        batch_id='clean-ecom-{{ ts_nodash.lower() }}-{{
task_instance.try_number }}',
        batch={
            'pyspark_batch': {
                'main_python_file_uri': 'gs://pub-sub-
cleandata-dag/scripts/clean_ecom2.py',
                'jar_file_uris': [
                    'gs://pub-sub-cleandata-dag/jars/spark-
3.5-spanner-1.4.0.jar'
                ],
            },
            'runtime_config': {
                'version': '2.2',
            },
        },
    )

```

```

        },
    )

# Task 2: BigQuery Federated Aggregation Query
run_bigquery_summary = BigQueryInsertJobOperator(
    task_id='run_bigquery_aggregation',
    configuration={
        "query": {
            "query": """
                INSERT INTO `snappy-mapper-498509-
e0.ecom_analytics2.daily_product_summary` (
                    product_id,
                    total_items_sold,
                    total_orders,
                    aggregated_at
                )
                SELECT
                    product_id,
                    SUM(quantity) AS total_items_sold,
                    COUNT(DISTINCT order_id) AS
total_orders,
                    CURRENT_TIMESTAMP() AS aggregated_at
                FROM EXTERNAL_QUERY(
                    "projects/snappy-mapper-498509-
e0/locations/asia-south1/connections/spanner-ecom-conn",
                    "SELECT order_id, product_id, quantity
FROM EcomOrders"
                )
                GROUP BY product_id;
            """,
            "useLegacySql": False,

```

```

    }

    },

)

# Dependency Order

run_dataproc_batch >> run_bigquery_summary

```

5.Verification Code in BigQuery

Run the following query in BigQuery Studio to inspect materialized summary metrics:

```

SELECT

    product_id,

    total_items_sold,

    total_orders,

    aggregated_at

FROM ` / snappy-mapper-498509-
e0.ecom_analytics.daily_product_summary`

ORDER BY total_items_sold DESC;

```

6.IAM Roles & Permissions Summary

Service / Integration	Principal / Service Account	Assigned IAM Roles	Purpose
Pub/Sub → Cloud Storage	Pub/Sub Service Agent service-54903645206@gcp-sa-	- roles/storage.objectCreator - roles/storage.legacyBucketReader	Allows Pub/Sub to write streaming JSON order files directly into the raw Bronze GCS bucket (pub-sub-rawdata).

Service / Integration	Principal / Service Account	Assigned IAM Roles	Purpose
	pubsub.iam.gserviceaccount.com		
Dataproc Serverless (PySpark)	Compute Engine Default Service Account 54903645206-compute@developer.gserviceaccount.com	- roles/dataproc.worker - roles/storage.objectAdmin - roles/spanner.databaseUser	Grants compute execution rights, allows reading raw JSON/writing Parquet in Silver GCS (pubsub-clean-data), and grants write/append access to Cloud Spanner (EcomOrders).
BigQuery External Link	BigQuery Connection Service Agent bqcx-...@gcp-sa-bigquery-connection.iam.gserviceaccount.com	roles/spanner.databaseReader	Allows BigQuery to query Cloud Spanner tables directly in real time via federated EXTERNAL_QUERY.
BigQuery Automation	BigQuery User / Service Account executing scheduled tasks	- roles/bigquery.dataEditor - roles/bigquery.connectionUser - roles/bigquery.jobUser	Grants permissions to execute scheduled queries, use the Spanner external connection, and append aggregated daily sales data into ecom_analytics.daily_product_summary.